

CODENTRY: EVIDENCE-FIRST PULL-REQUEST ANALYSIS

Literature Survey for Final Year Engineering Project — Updated Edition

Grounding differential static analysis, optional AI review, and empirical evaluation in 61 sources
(2014–2026)

Rushikesh Malgan · Vansh Mhatre · Yash Jadhav · Piyush Waghmare

Department of Computer Engineering, University of Mumbai — Batch 2027

Original edition: 26 August 2026 (30 papers). **This edition:** 24 September 2026 (61 sources).

Research cut-off for new material: **24 September 2026.**

Revision note. This edition was prepared with an AI assistant. Every reference was checked against arXiv's API or a publisher/organizer page on the research date, and numbers were taken from abstracts (not full texts) except where stated. The team should open each source before defending a claim from it. See `literature-survey-change-log-2026-09-24.md` for every change and correction.

Abstract

Codentry is a GitHub pull-request analysis system. Its current implementation performs *differential* static analysis (ESLint and a small hand-written Semgrep ruleset applied to the merge base and the head of a pull request, reporting only what the change introduces); an AI review signal is planned as an *optional* second signal, not as the product. Its research question is: *how do deterministic static analysis and LLM-based review differ in defect coverage and review noise on pull-request changes, and does combining their independent evidence improve useful defect detection without producing unacceptable false positives?*

This survey grounds that position in 61 sources: the 30 papers of the earlier edition (re-verified; author names, several numbers, and several attributions corrected) and 31 sources added on 24 September 2026 on static-analysis noise and actionability, change-level (differential) analysis, evaluation methodology, ground-truth construction, LLM-evaluation validity (non-determinism, self-preference, contamination), the security of analysis pipelines, and pull-request authorship. The survey does **not** claim that LLMs are better than static analysis, that static analysis is better than LLMs, or that combining them is proven better; the evidence it reviews is mixed, is largely preprint-based, and is often produced under evaluation designs whose ground truth is itself uncertain. It concludes that measurement — with defensible ground truth, per-signal metrics, repeated runs, and stated limitations — is the contribution a project of this size can make. Repository-level retrieval (RAG) and multi-agent review, prominent in the earlier edition, are demoted to background: the evidence does not make them prerequisites.

1. Introduction and Research Position

Code review is a widely used quality practice, but it costs human time and depends on context that a diff may not contain. Automated tools address parts of that cost. Static analyzers are deterministic and cheap but are known for false alarms and for reporting problems that were already in the code [33, 35]; LLM-based reviewers can comment on intent and semantics but are non-deterministic [46], can be biased when evaluated by models like themselves [43, 44], and are hard to evaluate without independent ground truth [41]. Industrial reports show real adoption alongside real drawbacks: in one deployment 73.8% of automated comments were resolved, yet pull-request closure time increased and developers reported faulty reviews, unnecessary corrections, and irrelevant comments [40].

Position of this project. Codentry is an *evidence-first* system: each finding carries the tool, rule, location, and code it refers to; findings are classified relative to the merge base (new, existing, fixed); and the research value lies in *measuring* the accuracy, overlap, and noise of each signal. It is not a claim about which signal is better. It is also not a generic "AI code reviewer": the AI signal is optional, gated behind an evaluation harness, and must be evaluated against ground truth that no language model produced.

What this edition changes. The earlier edition (26 August 2026) framed Codentry as an AI-powered reviewer with a static/AI split, repository-level RAG, and a planned evaluation harness, and claimed that the combination was unique within the corpus. This edition (i) reframes the position as above; (ii) reorganizes the themes around static-analysis noise, evaluation, ground truth, signal combination, pipeline security, and authorship; (iii) moves RAG and multi-agent review to background; (iv) weakens the uniqueness claim to what the reviewed sources support; and (v) corrects errors found while verifying the original 30 references (Section 2.3 and Appendix A).

2. Method, Corpus, and Verification

2.1 Corpus

The corpus has two parts. **Sources [1]–[30]** are the earlier edition's papers, chosen by the project team (not by a systematic search). **Sources [31]–[61]** were added on 24 September 2026 through 26 targeted web searches (listed in the change log) followed by page-level verification, on the topics in Section 2.2; they were selected for a clear connection to the research question, not because they mention AI or code. The search tool was US-only and search-engine-ranked, so recall is unknown. This is **not** a systematic review: there is no PRISMA-style protocol, inclusion/exclusion process, or double screening.

2.2 Search topics

LLM-based and automated code review; static analysis and its false positives; differential/change-level analysis and warning tracking; review noise and actionability; review benchmarks and datasets; ground-truth construction (mutation testing, SZZ, real-fault benchmarks); LLM evaluation methodology (non-determinism, self-preference, contamination, guidelines); AI-generated code and its review; finding localization; security of analysis and CI pipelines (configuration-as-code, prompt injection). Topics with no adequate source found are reported as such in Section 11.

2.3 Verification performed

Check	Result
All 30 original references exist (arXiv API, identifier and title)	30 / 30 (the Springer chapter [29] was confirmed through its Springer and institutional listings)
Original first-author names match arXiv	3 of 30 (2 fully: [25], [29]; 1 by surname only: [8]); 27 were wrong
Original numeric claims spot-checked against abstracts	matched for [8], [9], [11], [14], [15], [16], [20], [23], [24], [26], [28] and most others; the exceptions are listed below
New references [31]–[61]	metadata and abstract retrieved for all; publisher pages (ACM, IEEE, Springer) refused access, so bibliographic details were confirmed through Crossref instead: the DOIs of [29], [31], [32], [33], [34], [37], [40], [46], [47], [52], [53], [54], and [59] resolved to the expected title, authors, and venue on 2026-09-24. The DOI given in arXiv's comment field for [39] (10.1145/3832100) returned 404 in Crossref, and [48] has no Crossref record yet; both are flagged. FSE 2014 [53] and VL/HCC 2017 [31] were also read from the papers' own PDFs
Venue statuses	confirmed by Crossref where a DOI exists; taken from arXiv comment fields only for [39], [41], and [48] and marked "per arXiv metadata"

Corrections to the earlier edition that affect the argument (full list in Appendix A): *(i)* the earlier text attributed "dynamic runtime evaluation" as the key future direction to the SLR [30]; it appears in the benchmark survey [10]; *(ii)* the claim that [7] evaluates pgvector as a pragmatic choice is not supported by that paper's abstract and is withdrawn as unverified; *(iii)* MAF-CPR's [29] agents are a Repository Manager, PR Analyzer, Issue Tracker, and Code Reviewer, not the "change scope / semantic understanding / integration risk" agents the earlier text described; *(iv)* the timing figures attributed to [25] were wrong (the abstract reports a 33% reduction in review time and 87% in waiting time; more than 60% in median resolution time); *(v)* $\kappa = 0.75$ in [8] validates the LLM-as-judge framework, not the human annotation; *(vi)* "Terragni et al. [21]" is Duma et al.

2.4 Reading convention

For each source the survey separates *what the paper reports* (from its abstract), *what the authors claim*, and *what Codentry can reasonably infer*. Numbers are quoted from abstracts; interpretation is marked as such. Preprint status is stated wherever it matters.

3. Static Analysis in Code Review: Noise, Actionability, and Change-Level Analysis

What the papers report. Static analysis overlaps with human review only partly. On 274 reviewer comments from 92 GitHub pull requests in 23 Java repositories, PMD warnings overlapped with nearly 16% of comments, and the authors identified four further rules that could pre-empt reviewer effort [31] (the authors note that inter-rater reliability was not measured). At the change level, a study of 815 vulnerability-contributing commits in 92 C/C++ projects found that a single SAST tool produced warnings in the vulnerable functions of 52% of commits, that prioritizing changed functions with warnings improved precision by 12% and recall by 5.6%, that at least 76% of warnings in vulnerable functions were irrelevant to the commits, and that 22% of commits were undetected because of rule limitations [32]. For JavaScript, a study of nine tools on 957 Node.js vulnerabilities found that the three best combined detected up to 57.6% of them at a precision of 0.11%, and that many OWASP Top-10 vulnerabilities were detected by no tool [37].

Noise and actionability. False alarms are the recurring obstacle. A retrospective analysis showed that earlier "near-perfect" actionable-warning classifiers were inflated by data leakage and duplicated warnings, and that the heuristic *warning oracle* used to label ground truth (comparing a warning at one revision with a later reference revision) produced labels that disagree with human oracles [33]. A large Java dataset (NASCAR, over one million warnings) attempts to separate actionable from non-actionable warnings, motivated by "alert fatigue" [35] (preprint). Tracking warnings across commits is presented as necessary for letting developers concentrate on recent, actionable warnings rather than thousands of project-wide ones [34] (IEEE TSE listing; only the abstract was verified). Using LLMs to filter static alerts is an active line: at Tencent, hybrid LLM-plus-static techniques were reported to eliminate 94–98% of false positives with high recall on 433 alarms, at per-alarm costs of \$0.0011–\$0.12 and against 10–20 minutes of manual inspection per alarm [38]; agent-based filtering of SAST alerts removed most noise but could suppress true vulnerabilities and depended on the backbone model and the vulnerability class [39].

Reading across the papers. (1) Change-level framing is established for static analysis [32, 34]. (2) How a warning is *labeled* is itself a validity problem [33]. (3) The two LLM filtering studies treat the static tool as the primary signal and the LLM as a filter [38, 39]; they do not measure the two as independent reviewers.

Implication for Codentry. Differential (new / existing / fixed) classification is consistent with the change-level literature, and per-PR false-positive counting requires exactly the adjudicated labels that [33] and [41] warn are hard to obtain. The static arm should be reported with its known limits — six hand-written rules will probably detect little of what [37] shows tools miss — as a *baseline to be measured*, not a strong detector.

4. Evaluating AI Code Review: Benchmarks and Metrics

Landscape. A survey of 99 papers (58 pre-LLM, 41 LLM-era; 2015–2025) proposes a taxonomy of five domains and 18 tasks, reports a shift toward end-to-end generative review, and names dynamic runtime evaluation, broader task coverage, and taxonomy-guided fine-grained assessment as future directions [10]. An SLR of 119 papers on code-review automation catalogues tasks, techniques, datasets, and metrics [30].

Benchmarks and their reported findings. SWE-PRBench (350 PRs, human-annotated) reports that eight frontier models detect only 15–31% of human-flagged issues on the diff-only configuration; its LLM-as-judge framework was validated at $\kappa = 0.75$ [8]; a structured 2,000-token prompt outperformed a 2,500-token full-

context prompt (the "attention dilution" reading is the survey authors' interpretation of that result). SWR-Bench (1,000 manually verified PRs) reports ~90% agreement between its objective LLM-based evaluation and human judgment, that ACR tools were more adept at functional errors, and that a simple multi-review aggregation increased F1 by up to 43.67% [11]. c-CRAB generates executable tests from human reviews as a quality gate; the reviewed agents together solved about 40% of tasks, and agent reviews often considered different aspects than human reviews [9]. CodeFuse-CR-Bench (601 instances, 70 Python projects, nine problem domains) combines rule-based checks for location and syntax with model-based judgments of quality and finds no single LLM dominant [1]. ContextCRBench (67,910 entries) supports hunk-level assessment, **line-level defect localization**, and comment generation, and reports that textual context helped more than code context [42] (preprint). A small evaluation of GPT-4o and Gemini 2.0 Flash on 492 AI-generated code blocks reported 68.50% and 63.89% correctness classification with problem descriptions, with lower performance without them [26].

Practice. Industrial evaluations report both benefits and costs [40], and evaluating bot comments *automatically* is hard: LLM judges agreed with developer fixed/wontFix labels only moderately (0.44–0.62 across three models on 2,604 comments), and the authors argue that developer actions reflect workflow pressures as well as comment quality, so they are a weak proxy for ground truth [41]. A 2026 preprint comparing five models on 100 mutation-injected bugs and 50 real bug-fix PRs reports a large drop in F1 from synthetic to real cases and near-zero recall on performance bugs [55]; its arXiv identifier (2606) is inconsistent with its stated submission date (April 2026), it has seven pages and 13 references, and it is treated here as **low-confidence, illustrative only**.

Implication for Codentry. Report metrics separately (location accuracy, coverage, false positives per PR, duplicates, cost, latency), as [1] and [10] motivate; use deterministic location matching for the static arm; do not use developer actions or an LLM judge as sole ground truth [41]; and treat any single-source benchmark number as provisional.

5. Ground Truth and the Validity of LLM Evaluation

Constructing ground truth. Mutation-seeded defects scale but are proxies: a study of 357 real faults in five open-source programs found a statistically significant correlation between a test suite's mutant-detection rate and its real-fault-detection rate, independent of code coverage, while also revealing inherent limitations [53]; it studied test suites, not reviewers, so the result does not transfer automatically to review evaluation. For real defects in JavaScript, BugsJS provides 453 real, manually validated bugs from ten Node.js programs, each with bug report, failing tests, and fixing patch [54] (project site; the paper was not opened). Deriving bug-introducing commits automatically (SZZ) is noisy, and evaluations of SZZ variants rely on researchers' judgment; a developer-informed oracle built from commits that explicitly reference the introducing commit was proposed as a more reliable check [52].

Contamination. Benchmarks built from public repositories may overlap with training data. On SWE-Bench, one study reports that models identify buggy file paths from issue text alone with up to 76% accuracy, versus up to 53% on repositories outside the benchmark, and much higher verbatim similarity on SWE-Bench Verified (up to 35% versus 18% 5-gram accuracy) [49]; another finds solution leakage in 32.67% of successful patches, weak tests in 31.08%, a resolution rate that fell from 12.47% to 3.97% after filtering, and over 94% of issues created before the model's knowledge cutoff [50]. A survey of static-to-dynamic benchmarking notes the lack of standardized criteria for dynamic benchmarks [51]. All three are preprints.

Evaluator and model bias. LLM judges show self-preference: GPT-4 exhibited significant self-preference, and the effect was linked to lower perplexity (familiarity) of the text [43] (workshop paper); across 20 LLMs, advanced capability was often uncorrelated, or even negatively correlated, with *low* self-preference bias (more capable judges were not less biased), and a structured evaluation strategy reduced the bias by 31.5% on average [44] (preprint). For code specifically, GPT-3.5 and GPT-4 were unable to precisely identify

vulnerabilities in the code they generated and performed poorly when repairing self-produced code, which the authors describe as self-repair "blind spots" [45] (preprint).

Non-determinism and reporting. For 829 code-generation problems, ChatGPT returned zero equal test outputs across requests for 75.76%, 51.00%, and 47.56% of tasks on three benchmarks; temperature 0 did not guarantee determinism [46]. Threats from closed-source model drift, training-data leakage, and irreproducibility motivate guidelines [47]; a 22-author taxonomy of study types and eight guidelines recommend, among others, reporting model versions and configurations, documenting prompt and system design, reporting interaction traces, validating LLM outputs against human judgment, and including an open LLM as a baseline [48].

Implication for Codentry. *Ground truth must be independent of the model being evaluated:* executable tests first; then mutation-seeded defects (reported by stratum, with the proxy caveat); then verified historical defects; then independent human labels with agreement reported. An LLM must never be the sole ground truth for LLM output [43–45]. Any AI arm should be run repeatedly and reported with variance [46], pin and record model versions, prompts, and parameters [47, 48], prefer post-cutoff or private/synthetic cases where licensing and data policy allow [49–51], and be compared against an open-model baseline where hardware permits [48].

6. Combining Static and LLM Signals

Reported comparisons. A comparison of 15 SAST tools and 12 open-source LLMs on Java, C, and Python repositories reports that SAST tools had low detection rates with relatively low false positives, that LLMs detected up to 90–100% of vulnerabilities with high false positives, and that ensembling mitigated some of each drawback [36] (preprint). An agentic secure-code reviewer with security-focused semantic memory reports at least a 153% relative improvement over a static-LLM baseline, a multi-agent reviewer, and SAST tools in comments with correct localization, vulnerability type, and relevance, and a 54% validation rate by security engineers in shadow deployment [15]. Multi-review aggregation raised F1 by up to 43.67% in [11]. In a comment-injection study, cross-referencing an LLM reviewer's verdict with static analysis was the best of four defenses (96.9% detection, recovering 47% of baseline misses) [58] (preprint, 100-sample benchmark).

The independence argument. An information-theoretic paper proves that combining agents with different detection patterns finds more bugs than one agent under conditional independence, and reports agent correlation $\rho = 0.05\text{--}0.25$ on 99 verified samples [14]. Those agents are all LLM-based specialists (correctness, security, performance, style). Whether a *static tool* and an *LLM reviewer* are similarly independent on pull requests is an empirical question this project intends to measure, not a result of [14].

Design consequence. If the AI signal is shown the static tool's findings (the earlier PRD design), the two signals are no longer independent and overlap/independence cannot be measured. The research arms must therefore run separately on the same inputs and be combined only afterwards.

Implication for Codentry. Treat "combination improves useful detection" as a hypothesis with a test: per-arm coverage, overlap, unique catches, and false positives per PR, with paired comparison and intervals.

7. Security of Analysis and Review Pipelines

Code produced by AI agents. On 186 feature-request tasks, 57% of SWE-Agent-with-Claude-4-Sonnet solutions were functionally correct but only 11.8% were secure, and simple security hints did not fix this [16]; LLM agents achieved at most 18.0% success on proof-of-concept generation and 34.0% on vulnerability patching in SEC-bench [17].

Attack surface of AI reviewers and agents. A NIST-RFI response describes agent architectures as changing assumptions about code-data separation and authority boundaries, emphasizing indirect prompt injection

and confused-deputy behavior [18]. Across three environments and 13 architectural configurations, multi-agent architectures were more vulnerable than standalone agents in most configurations, with attack success varying up to 3.8× [19]. In a coder-reviewer-tester system, adding a security-analysis agent improved resilience, but poisonous few-shot examples raised attack success against that agent from 0% to 71.95% [20].

Pipelines and configuration. GitInject evaluates prompt-injection attacks in live GitHub workflows using AI agents and documents eleven named attacks spanning config-file injection, credential exfiltration, judgment manipulation, and availability; all tested providers were susceptible to at least one class in default configuration, and the most critical vulnerabilities were *structural* — how CI/CD handles credentials and configuration files — rather than model-specific [56]. A taint study of 1,033 AI-assisted GitHub Actions and 13,392 workflows in 10,792 repositories reported 519 potential injection vulnerabilities, 496 confirmed exploitable, and 343 previously unknown [57]. Earlier work applied a security-assessment methodology to GitHub Actions workflows on 50 open-source projects and reported 24,905 issues [59]. GitHub's own guidance recommends avoiding `pull_request_target` and `workflow_run` with untrusted pull requests and treating untrusted input as data [W1]. In contrast to alarming demonstrations, a controlled study of comment-based attacks on eight models over 9,366 trials found small, statistically non-significant effects on vulnerability-detection accuracy [58]; the threat models differ (workflow-level compromise versus persuading a model through comments).

Implication for Codentry. The finding that configuration and credential handling are the structural weakness [56] matches what Phase 0 of this project found in *its own static-analysis pipeline*: a pull request's ESLint configuration and parser executed on the analysis server, and the analysis subprocesses inherited every server secret (executed proofs-of-concept are recorded in the project's Phase 0 report). None of the sources reviewed addressed the execution of static-analysis-tool configuration (ESLint/Semgrep) specifically; that absence reflects the limited coverage of this survey, not proof that the topic is unstudied. Any future AI arm must run without repository credentials and treat repository text as data, consistent with [18], [56], and [57]. The earlier edition's suggestion that Semgrep could be configured to screen for prompt injection is not supported by any reviewed source and is withdrawn.

8. Human-AI Interaction, Authorship, and Pull-Request Behavior

Review of AI-authored PRs. Using the AIDev dataset, most AI-generated PRs received no review; when reviewed, review was dominated by AI agents, with human involvement expressed through agent steering; human-authored PRs were more likely to receive human-only review [21]. AIDev itself contains over 456,000 PRs by five agents across 61,000 repositories and 47,000 developers and reports that agents are faster but their PRs are accepted less often [60]. A dataset of 248,641 AI-attributed PRs with AI-attributed reviews found cross-product AI-to-AI review in about 1.6% of identified agent-authored PRs, growing by more than two orders of magnitude from 2025-Q1 to 2025-Q3 [61] (preprint). Across 7,156 PRs, task type (documentation 82.1% accepted versus new features 66.1%) produced a gap exceeding typical inter-agent variance for most tasks, and no single agent was best across all nine task categories [22].

Effects of LLM assistance and field evidence. For 25,473 PRs from 9,254 projects, GPT-assisted PRs showed more than a 60% reduction in median resolution time (9 versus 23 hours), a 33% reduction in review time, and an 87% reduction in waiting time before acceptance [25] (the earlier edition misreported these). A field study at WirelessCar found that AI-led reviews were preferred overall, conditional on reviewers' familiarity with the code and PR severity, and recorded concerns about false positives and trust [27]. ARCTIC reports production results (intent prediction F1 0.86; drift detection QWK 0.907; code spotlight 2.4× quality at 5× fewer tokens; 90.2% approval; zero defects attributed to self-reviewed diffs since launch) [28] — authors' self-reported figures from one organization.

Authorship as an experimental variable. The sources show that review dynamics and acceptance differ by authorship and by task type [21, 22, 60, 61], which makes authorship a *plausible* stratification variable for

evaluation. No reviewed source reports **defect-detection accuracy** of reviewers stratified by authorship, and task type can confound authorship [22]; attribution of authorship also depends on dataset heuristics. In this project authorship is therefore a candidate variable to record and stratify when observable, not a hypothesis with literature support.

Implication for Codentry. Do not build evaluation sets exclusively from AI-authored PRs without stating the review dynamics that produced their labels [21]; weight reporting by task type where possible [22]; and do not treat resolution or acceptance as ground truth for correctness [41].

9. Background Retained from the Earlier Edition: Context Retrieval and Multi-Agent Review

The earlier edition made repository-level RAG (Theme A) and multi-agent architectures (Theme C) central. The evidence does not make either a prerequisite, so this edition keeps them as background and future work.

Context and retrieval. Existing benchmarks miss real-world context; CodeFuse-CR-Bench frames a "reality gap" [1]. ContextBench (1,136 tasks, 66 repositories, eight languages) reports that sophisticated scaffolding yields only marginal gains in retrieval, LLMs favor recall over precision, and explored and utilized context differ substantially [2]. Agent Retrieval Bench (427 samples, 25 repositories) reports that no single retrieval family dominates, that thresholds calibrated on counterfactual controls did not help on natural no-gold cases, and that logged trajectories missed every gold file on 27–35% of samples [3]. RAG for review-comment generation improved exact match by up to +1.67% and BLEU by up to +4.25% on the Tufano et al. benchmark, with performance improving as more exemplars were retrieved [4]; a second study found that top-1 retrieval worked best and more retrieval hurt [5]. Even with oracle context the best model reached 69.1% Pass@1 on cross-file output prediction, and models showed partial reliance on memorization [6]. A survey organizes retrieval-augmented code generation for repository-level tasks [7] (its abstract does not mention pgvector, contrary to the earlier edition). LAURA reports that review comments were completely correct or helpful in 42.2% (ChatGPT-4o) and 40.4% (DeepSeek v3) of cases, with all three components contributing [23]; Sphinx reports up to 40% improvement in checklist coverage over baselines [24]. **Reading:** retrieval sometimes helps and can hurt; none of these results is on JavaScript/TypeScript pull-request *defect detection* with independent ground truth, and none shows RAG is needed for Codentry's research question.

Multi-agent review. RepoReviewer is a systems contribution without a benchmark-superiority claim [12]; SWE-Review reports that a generate-review-revise loop improves PRs and enables test-time scaling [13]; MAF-CPR uses four specialized agents and reports outperforming GPT-3.5, GPT-4, and Claude-3-Sonnet on four tasks [29] (peer-reviewed chapter). Security studies show multi-agent architectures can be more vulnerable [19]. **Reading:** multi-agent designs raise cost, complexity, and attack surface; measuring the independence of two signals [14] does not require them.

10. Evidence Tables

Table 1. Sources added in this edition (31). Numbers are from abstracts unless stated.

Paper	Year	Area	Method	Key finding	Relevance to Codentry	Limitation
Singh et al. [31]	2017	Static analysis vs. review	PMD on 92 PRs (23 Java repos), manual overlap with 274 comments	PMD overlapped ~16% of comments	Static/human overlap is partial	Java only; inter-rater reliability not measured
Charoenwet et al. [32]	2024	Change-level SAST	815 vulnerability-contributing commits, 92 C/C++ projects	52% of commits had warnings in vulnerable functions; ≥76% of such warnings irrelevant; 22% undetected	Change-level evaluation; noise	C/C++; SAST only
Kang et al. [33]	2022	Warning actionability / ground truth	Retrospective re-analysis	Leakage and duplication inflated results; warning oracle disagreed with humans	Ground-truth caution for warnings	Java-centric prior work
Li & Yang [34]	2024	Warning tracking	Tracking approaches across commits (abstract only verified)	Tracking is critical for actionable focus	Supports differential framing	Details not verified; venue via IEEE listing
Kószó et al. [35]	2025	Actionable warnings dataset	Methodology + >1M Java warnings	Distinguishes (non-)actionable; alert fatigue	Dataset/label approach	Preprint; Java
Zhou et al. [36]	2024	SAST vs. LLM	15 SAST tools vs. 12 LLMs; Java, C, Python	SAST: low detection/low FP; LLM: high detection/high FP; ensembling helps	Direct static-vs-LLM evidence	Vulnerability detection, not PR review; preprint
Brito et al. [37]	2023	JS static tools	9 tools, 957 Node.js vulnerabilities	Best 3 combined ≤57.6% detection at 0.11% precision	Expect low static recall on JS	Vulnerabilities only; older tools
Du et al. [38]	2026	LLM FP reduction	Industrial, 433 alarms (Tencent)	Hybrid LLM+static removed 94–98% of FPs	LLM as filter	One company; three bug types; preprint
Xiong & Zhang [39]	2026	LLM agents filter SAST FPs	3 agent frameworks, OWASP + Java + OSS-Fuzz	Large noise reduction; can suppress true vulns; backbone-dependent	Trade-offs of filtering	SAST alerts only
Cihan et al. [40]	2025	Industrial ACR	4,335 PRs, 238 practitioners	73.8% comments resolved; closure time rose 5h52m→8h20m; faulty/irrelevant comments	Noise/cost in practice	One organization
Karakaya et al. [41]	2026	Evaluating ACR bots	2,604 comments, 3 LLM judges	Agreement 0.44–0.62 with developer labels	Developer labels ≠ ground truth	One company
Hu et al. [42]	2025	Benchmark, localization	67,910 entries; line-level localization task	Text context helped more than code context	Localization metric design	Preprint
Wataoka et al. [43]	2024	Self-preference	New metric; perplexity analysis	GPT-4 self-preference; tied to familiarity	Same-model evaluation bias	Dialogue-system judging; workshop
Yang et al. [44]	2026	Self-preference	20 LLMs, equal-quality pairs	Capability did not imply low SPB; −31.5% with mitigation	Judge selection	Preprint
Gong et al. [45]	2024	Same-model blind spots	4,900 code samples	Models poor at finding/repairing own vulnerabilities	Correlated errors	Older models; preprint
Ouyang et al. [46]	2025	LLM non-determinism	829 tasks, 3 benchmarks	75.76%/51.00%/47.56% zero-equal outputs	Repeat AI runs	ChatGPT, code generation
Sallou et al. [47]	2024	Threats to validity	Position paper	Closed models, leakage, irreproducibility	Reporting rules	Position, no data
Baltes et al. [48]	2025	Guidelines	22-author taxonomy + 8 guidelines	Report versions/prompts; open-LLM baseline; validate outputs	Protocol for AI arm	Guidance, not evidence
Liang et al. [49]	2025	Contamination	Diagnostic tasks on SWE-Bench	Up to 76% path accuracy vs. 53% off-benchmark	Contamination risk	Issue-resolution, not review; preprint
Aleithan et al. [50]	2024	Benchmark quality	Manual screening of SWE-bench	32.67% leakage; 31.08% weak tests; 12.47%→3.97%	Label/benchmark quality	One agent; preprint
Chen et al. [51]	2025	Contamination survey	Survey of static→dynamic benchmarks	No standard for dynamic benchmarks	Mitigation ideas	Survey; preprint

Paper	Year	Area	Method	Key finding	Relevance to Codentry	Limitation
Rosa et al. [52]	2021	Ground truth (SZZ)	Developer-informed oracle	Manual SZZ evaluation by non-developers is weak	Avoid naive SZZ	SZZ scope
Just et al. [53]	2014	Mutation realism	357 real faults, 5 programs, 321 kLOC	Mutant detection correlates with real-fault detection	Mutants as proxy	Test suites, not reviewers; Java
Gyimesi et al. [54]	2019	Real JS defects	453 bugs, 10 Node.js programs, tests + patches	Executable ground truth for JS	Dataset for real-defect stratum	Popular repos → contamination; project site read
Kumar et al. [55]	2026	LLM review comparison	100 mutants + 50 real bug-fix PRs, 5 models	Large synthetic→real drop	Mutant-realism concern	Low confidence: 7 pages; ID/date mismatch
Isbarov et al. [56]	2026	CI prompt injection	Live GitHub workflows, 4 providers	11 attacks; structural credential/config issues	Config/credential trust	Preprint
Wang et al. [57]	2026	Agentic workflow injection	13,392 workflows; taint analysis	496 confirmed exploitable of 519	Untrusted event text → agents	Preprint
Thornton [58]	2026	Comment attacks	8 models, 9,366 trials	Small non-significant effects; static cross-check best defense	Static as safeguard	100 samples; preprint
Benedetti et al. [59]	2022	Workflow security	Tool on 50 projects	24,905 issues	CI config risk	Workshop paper
Li et al. [60]	2025	AI-authored PRs dataset	AIDev: >456k PRs	Agents faster, accepted less	Authorship variable	Attribution by heuristics
Selvanayagam & Ghaleb [61]	2026	AI-to-AI review	248,641 PRs	Cross-product review ~1.6% of agent PRs, growing fast	Closed-loop AI review	Preprint

Table 2. Codentry design decisions and the strength of the evidence found.

Design decision	Sources	Strength / caveat
Change-level (differential) reporting	[32], [34], [31]	Moderate for static analysis; no PR-level LLM-vs-static differential study found
Static analysis as an independent baseline	[36], [37], [32]	Baseline recall may be low; informative, not "good"
AI as an optional second signal	[36], [38], [39], [15], [11]	Mixed: high LLM detection with high FP; filtering helps; task-dependent
Evaluation as the core function	[10], [33], [41]	Strong motivation; ground truth is the recurring obstacle
Ground truth from tests/mutants/real bugs/humans	[52], [53], [54], [9]	Moderate; mutants are proxies
False-positive / noise measurement per PR	[33], [35], [40], [41]	Strong need; per-PR measurement not standardized in sources found
Finding identity / location stability	[34], [42]	Limited evidence found
Same-model evaluation bias	[43], [44], [45]	Moderate; measured mostly outside code review
Repeated runs and version reporting	[46], [47], [48]	Strong
Contamination controls	[49], [50], [51]	Strong for issue-resolution benchmarks; assumed to transfer
Security of the analysis pipeline itself	[56], [57], [59], [W1], [18]–[20]	Growing (mostly 2026 preprints); ESLint/Semgrep config execution not found
Authorship as a variable	[21], [22], [60], [61]	Plausible; detection accuracy by authorship not found
RAG not required	[3], [5], [6]	Mixed evidence; not disproved
Multi-agent not required	[12], [13], [19], [29]	Cost/attack-surface trade-offs

11. Research Gaps (stated with care)

The earlier edition said no paper in its corpus achieved a combination of properties. This edition replaces uniqueness claims with what the reviewed sources support. "Limited evidence" below means *in the sources reviewed*, which are not a systematic sample.

- Differential / new-findings-only evaluation.** Change-level and warning-tracking work exists for static analysis [32, 34]. *Limited evidence was identified* of PR-level comparisons of static and LLM review that classify findings relative to the merge base. **Under-explored in the sources reviewed.**
- Independent comparison of static and LLM signals.** *Partly covered* for vulnerability detection [36] and for LLMs as filters of static alerts [38, 39]. The earlier claim that no paper separates the layers was too strong; [36] compares them. *Limited evidence* for PR-review-comment comparison with independent ground truth on JavaScript/TypeScript.
- Review noise and false-positive measurement.** Widely recognized [33, 35, 40, 41]; *limited evidence* of a common per-PR noise metric across signals.
- Finding identity and location stability across revisions.** Warning tracking [34] and line-level localization [42] touch it; *limited evidence* on identity of LLM-generated findings across revisions.
- Reproducible PR-level evaluation.** Guidance exists [46–48]; how far code-review benchmarks follow it was **not assessed**.
- Ground-truth quality.** A recurring theme [33, 41, 52, 53]; supported.
- Human-authored versus AI-generated PRs.** Review dynamics and acceptance are studied [21, 22, 60, 61]; *limited evidence* of defect-detection accuracy stratified by authorship.
- Security of code-review automation itself.** Supported and growing [56–59, 18–20]; focused on AI agents in CI/CD.

9. **Configuration-as-code trust boundaries.** Supported for CI/CD and agent workflows [56, 57, 59, W1]; *no source reviewed addressed static-analysis-tool configuration execution* (ESLint/Semgrep) specifically.

No claim of the form "no one has done this" is made.

12. Threats to Validity

1. **Preprint bias.** Of the 30 earlier sources, 29 are arXiv preprints and one is a peer-reviewed chapter [29]. Of the 31 added, 16 carry a venue (VL/HCC 2017; ISSTA 2024 and 2026; ICSE 2021, 2022, and 2025 SEIP; ICSE-NIER 2024; FSE 2014; ICST 2019; IEEE TSE; IEEE Trans. Reliability; TOSEM; Empirical Software Engineering; EASE 2026; SCORED 2022; and one NeurIPS workshop paper) — three of those ([39], [41], [48]) from arXiv metadata only — and 15 are preprints. Findings resting on a single preprint are provisional.
2. **Selection bias.** The original 30 were team-selected; the 31 additions came from 26 searches through one search tool. There is no systematic protocol.
3. **Verification depth.** Abstracts were read, not full texts (exceptions: the first pages of [31] and [53]). Claims beyond abstracts are flagged.
4. **Assistant-prepared update.** The team must independently open each source before relying on it.
5. **Identifier/date anomaly.** [55]'s identifier month and stated submission date disagree; it is low-confidence and not load-bearing.
6. **Version and title drift.** Titles vary between versions of some preprints ([51], [58]); this edition cites the titles returned by arXiv on the research date.
7. **LLM-as-judge uncertainty and same-model bias** [8, 11, 41, 43–45] remain relevant to every LLM-evaluated result in the corpus; the project's own evaluation avoids model-produced ground truth.
8. **Language and repository bias.** Several sources are Python, Java, or C/C++ [1, 32, 36]; ESLint results on JavaScript/TypeScript cannot be inferred from them.
9. **Contamination** of any public-repository evaluation [49–51].
10. **Recency.** The corpus concentrates on 2024–2026; older foundational work is only partly represented ([31], [53], [59]).

13. Conclusion

The reviewed sources support a cautious, measurement-centered position. Change-level analysis and actionability are established concerns in static analysis, and the false-alarm problem is real [32–35]. LLM-based review shows promise and cost in practice [40] and is hard to evaluate without independent ground truth [41]; LLM evaluation carries specific validity threats — non-determinism, self-preference, contamination, and reporting gaps [43–51]. Comparisons of static analysis and LLMs report a trade-off between detection and false positives and suggest — but do not establish — that combining them helps [36]. The security of pipelines that ingest repository content is a live problem, and its structural causes lie in configuration and credential handling [56, 57], the same class of weakness Phase 0 found and fixed in Codentry's static pipeline. Retrieval and multi-agent designs are not prerequisites.

Codentry's contribution, therefore, is not a claim that one signal wins. It is a reproducible framework and a first set of measurements: differential static analysis as an independent baseline, an optional AI arm evaluated against ground truth no model produced, per-signal metrics with intervals, and explicit limitations. No experiment has been run at the time of this edition.

References

Sources [1]–[30] (earlier edition; author names corrected on 24 September 2026)

- [1] H. Guo, X. Zheng, Z. Liao, H. Yu, P. Di, Z. Zhang, and H.-N. Dai, "CodeFuse-CR-Bench: A Comprehensiveness-aware Benchmark for End-to-End Code Review Evaluation in Python Projects," arXiv:2509.14856, 2025.
- [2] H. Li, L. Zhu, B. Zhang, R. Feng, J. Wang, Y. Pan, E. T. Barr, F. Sarro, Z. Chu, and H. Ye, "ContextBench: A Benchmark for Context Retrieval in Coding Agents," arXiv:2602.05892, 2026.
- [3] B. Qin and Y. Xie, "Agent Retrieval Bench: Evaluating Repository Context Retrieval for Coding Agents," arXiv:2607.24882, 2026.
- [4] H. Hong and J. Baik, "Retrieval-Augmented Code Review Comment Generation," arXiv:2506.11591, 2025.
- [5] Q. Meng, X. Zhang, Z. Ren, and J. Visser, "When More Retrieval Hurts: Retrieval-Augmented Code Review Generation," arXiv:2511.05302, 2025.
- [6] Y. Wang, S. Wang, Y. Wang, B. Zhang, D. Guo, J. Chen, and Z. Zheng, "RepoReasoner: Evaluating Repository-Level Code Reasoning Ability of Long-Context Language Models," arXiv:2607.25996, 2026.
- [7] Y. Tao, Y. Li, Y. Qin, and Y. Liu, "Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches," arXiv:2510.04905, 2025.
- [8] D. Kumar, "SWE-PRBench: Benchmarking AI Code Review Quality Against Pull Request Feedback," arXiv:2603.26130, 2026.
- [9] Y. Zhang, Z. Pan, I. N. B. Yusuf, H. Ruan, R. Shariffdeen, and A. Roychoudhury, "Code Review Agent Benchmark," arXiv:2603.23448, 2026 (c-CRAB).
- [10] T. I. Khan, S. Wang, H. Zhang, and T.-H. Chen, "A Survey of Code Review Benchmarks and Evaluation Practices in Pre-LLM and LLM Era," arXiv:2602.13377, 2026.
- [11] Z. Zeng, R. Shi, K. Han, Y. Li, K. Sun, Y. Wang, Z. Yu, R. Xie, W. Ye, and S. Zhang, "SWR-Bench: Assessing LLM Performance in Real-World Code Review Comment Generation," arXiv:2509.01494, 2025.
- [12] P. Zhang, "RepoReviewer: A Local-First Multi-Agent Architecture for Repository-Level Code Review," arXiv:2603.16107, 2026.
- [13] R. Wang, J. Chen, S. Wang, C. Tao, S. Yang, Y. Jiang, K.-H. Yap, L. Shang, X. Li, and H. Bai, "SWE-Review: Closing the Loop on Issue Resolution with Agentic Code Review," arXiv:2607.06065, 2026.
- [14] S. Rajan, "Multi-Agent Code Verification via Information Theory," arXiv:2511.16708, 2025.
- [15] W. Charoenwet, K. Tantithamthavorn, P. Thongtanunam, H. Y. Lin, M. Jeong, and M. Wu, "AgenticSCR: An Autonomous Agentic Secure Code Review for Immature Vulnerabilities Detection," arXiv:2601.19138, 2026.
- [16] S. Zhao, D. Wang, K. Zhang, J. Luo, Z. Li, and L. Li, "Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks," arXiv:2512.03262, 2025.
- [17] H. Lee, Z. Zhang, H. Lu, and L. Zhang, "SEC-bench: Automated Benchmarking of LLM Agents on Real-World Software Security Tasks," arXiv:2506.11791, 2025.
- [18] N. Li, K. Zhang, K. Polley, and J. Ma, "Security Considerations for Artificial Intelligence Agents," arXiv:2603.12230, 2026.
- [19] B. Hagag, W. L. Anderson, C. Schroeder de Witt, and S. Scheffler, "Architecture Matters for Multi-Agent Security," arXiv:2604.23459, 2026.
- [20] B. Bowers, S. Khapre, and J. Kalita, "Analyzing Code Injection Attacks on LLM-based Multi-Agent Systems in Software Development," arXiv:2512.21818, 2025.

- [21] K. Duma, P. Wróblewski, J. Bobińska, J. Winiarska, and P. Przymus, "These Aren't the Reviews You're Looking For: How Humans Review AI-Generated Pull Requests," arXiv:2605.02273, 2026.
- [22] G. Pinna, J. Gong, D. Williams, and F. Sarro, "Comparing AI Coding Agents: A Task-Stratified Analysis of Pull Request Acceptance," arXiv:2602.08915, 2026.
- [23] Y. Zhang, Y. Zhang, Z. Sun, Y. Jiang, and H. Liu, "LAURA: Enhancing Code Review Generation with Context-Enriched Retrieval-Augmented LLM," arXiv:2512.01356, 2025.
- [24] D. Zhang, S. Zhang, Z. Jin, J. Luo, S. Fu, and E. Nallipogu, "Sphinx: Benchmarking and Modeling for LLM-Driven Pull Request Review," arXiv:2601.04252, 2026.
- [25] A. Collante, S. Abedu, S. Khatoonabadi, A. Abdellatif, E. Alor, and E. Shihab, "The Impact of Large Language Models (LLMs) on Code Review Process," arXiv:2508.11034, 2025.
- [26] U. Cihan, A. İçöz, V. Haratian, and E. Tüzün, "Evaluating Large Language Models for Code Review," arXiv:2505.20206, 2025.
- [27] F. S. Aðalsteinsson, B. B. Magnússon, M. Milicevic, A. N. Davidsson, and C.-H. Cheng, "Rethinking Code Review Workflows with LLM Assistance: An Empirical Study," arXiv:2505.16339, 2025.
- [28] C. Maddila, M. Rashik, E. M. Khan, S. Jha, J. Saindon, N. Nagappan, and P. C. Rigby, "From Code Review to Code Critique: Intent, Drift, and Spotlight for AI-Generated Diffs at Scale," arXiv:2607.29516, 2026.
- [29] F. Han, J. Peng, W. Wang, and X. Xia, "MAF-CPR: LLM-Based Multi-agent Framework for Complex Pull Request Review in GitHub," in *Advanced Intelligent Computing Technology and Applications (ICIC 2025)*, Springer Nature Singapore, 2025, pp. 247–257, DOI 10.1007/978-981-95-0020-8_21 (confirmed via Crossref).
- [30] R. Tufano and G. Bavota, "Automating Code Review: A Systematic Literature Review," arXiv:2503.09510, 2025.

Sources [31]–[61] (added 24 September 2026)

- [31] D. Singh, V. R. Sekar, K. T. Stolee, and B. Johnson, "Evaluating how static analysis tools can reduce code review effort," in *Proc. IEEE Symp. Visual Languages and Human-Centric Computing (VL/HCC)*, Oct. 2017, DOI 10.1109/VLHCC.2017.8103456.
- [32] W. Charoenwet, P. Thongtanunam, V.-T. Pham, and C. Treude, "An Empirical Study of Static Analysis Tools for Secure Code Review," in *Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2024, DOI 10.1145/3650212.3680313; arXiv:2407.12241.
- [33] H. J. Kang, K. L. Aw, and D. Lo, "Detecting False Alarms from Automatic Static Analysis Tools: How Far are We?," in *Proc. 44th Int. Conf. Software Engineering (ICSE)*, 2022, DOI 10.1145/3510003.3510214; arXiv:2202.05982.
- [34] J. Li and J. Yang, "Tracking the Evolution of Static Code Warnings: the State-of-the-Art and a Better Approach," *IEEE Trans. Software Engineering*, vol. 50, no. 3, pp. 534–550, Mar. 2024, DOI 10.1109/TSE.2024.3358283 (confirmed via Crossref); arXiv:2210.02651.
- [35] D. Kószó, T. Aladics, R. Ferenc, and P. Hegedűs, "A Large-Scale Collection Of (Non-)Actionable Static Code Analysis Reports," arXiv:2511.10323, 2025 (preprint).
- [36] X. Zhou, D.-M. Tran, T. Le-Cong, T. Zhang, I. C. Irsan, J. Sumarlin, B. Le, and D. Lo, "Comparison of Static Application Security Testing Tools and Large Language Models for Repo-level Vulnerability Detection," arXiv:2407.16235, 2024 (preprint; venue not stated in the retrieved metadata).
- [37] T. Brito, M. Ferreira, M. Monteiro, P. Lopes, M. Barros, J. F. Santos, and N. Santos, "Study of JavaScript Static Analysis Tools for Vulnerability Detection in Node.js Packages," *IEEE Trans. Reliability*, vol. 72, no. 4, pp. 1324–1339, Dec. 2023, DOI 10.1109/TR.2023.3286301; arXiv:2301.05097.

- [38] X. Du, J. Feng, Y. Zou, W. Xu, J. Ma, W. Zhang, S. Liu, X. Peng, and Y. Lou, "Reducing False Positives in Static Bug Detection with LLMs: An Empirical Study in Industry," arXiv:2601.18844, 2026 (preprint).
- [39] Y. Xiong and T. Zhang, "Sifting the Noise: A Comparative Study of LLM Agents in Vulnerability False Positive Filtering," in *Proc. 35th ACM SIGSOFT ISSTA*, 2026 (per arXiv metadata; the DOI given there, 10.1145/3832100, did not resolve in Crossref on 2026-09-24 — unverified); arXiv:2601.22952.
- [40] U. Cihan, V. Haratian, A. İçöz, M. K. Gül, Ö. Devran, E. F. Bayendur, B. M. Uçar, and E. Tüzün, "Automated Code Review In Practice," in *Proc. 47th IEEE/ACM Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025, pp. 425–436, DOI 10.1109/ICSE-SEIP66354.2025.00043 (confirmed via Crossref); arXiv:2412.18531.
- [41] V. Karakaya, U. B. Torun, B. M. Uçar, and E. Tüzün, "Understanding the Limits of Automated Evaluation for Code Review Bots in Practice," EASE 2026 (per arXiv comment); arXiv:2604.24525.
- [42] R. Hu, X. Wang, X.-C. Wen, Z. Zhang, B. Jiang, P. Gao, C. Peng, and C. Gao, "Benchmarking LLMs for Fine-Grained Code Review with Enriched Context in Practice," arXiv:2511.07017, 2025 (preprint).
- [43] K. Wataoka, T. Takahashi, and R. Ri, "Self-Preference Bias in LLM-as-a-Judge," NeurIPS 2024 Safe Generative AI Workshop; arXiv:2410.21819.
- [44] J. Yang, Z. Hu, C. Qiu, Z. Deng, X. Jiao, and T. Zhou, "Quantifying and Mitigating Self-Preference Bias of LLM Judges," arXiv:2604.22891, 2026 (preprint).
- [45] J. Gong, N. Duan, Z. Tao, Z. Gong, Y. Yuan, and M. Huang, "How Well Do Large Language Models Serve as End-to-End Secure Code Agents for Python?," arXiv:2408.10495, 2024 (revised 2025; preprint).
- [46] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, "An Empirical Study of the Non-determinism of ChatGPT in Code Generation," *ACM Trans. Software Engineering and Methodology*, vol. 34, no. 2, Jan. 2025, DOI 10.1145/3697010; arXiv:2308.02828.
- [47] J. Sallou, T. Durieux, and A. Panichella, "Breaking the Silence: the Threats of Using LLMs in Software Engineering," in *Proc. ICSE 2024, New Ideas and Emerging Results track*, DOI 10.1145/3639476.3639764.
- [48] S. Baltes *et al.* (22 authors), "Guidelines for Empirical Studies in Software Engineering involving Large Language Models," *Empirical Software Engineering* (accepted, per arXiv; no Crossref record found on 2026-09-24); arXiv:2508.15503.
- [49] S. Liang, S. Garg, and R. Zilouchian Moghaddam, "The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason," arXiv:2506.12286, 2025 (preprint).
- [50] R. Aleithan, H. Xue, M. M. Mohajer, E. Nnorom, G. Uddin, and S. Wang, "SWE-Bench+: Enhanced Coding Benchmark for LLMs," arXiv:2410.06992, 2024 (preprint).
- [51] S. Chen *et al.*, "Recent Advances in Large Language Model Benchmarks against Data Contamination: From Static to Dynamic Evaluation," arXiv:2502.17521, 2025 (preprint; the arXiv metadata spells "Langauge" and another index lists a different title for this identifier).
- [52] G. Rosa, L. Pascarella, S. Scalabrino, R. Tufano, G. Bavota, M. Lanza, and R. Oliveto, "Evaluating SZZ Implementations Through a Developer-informed Oracle," in *Proc. 43rd IEEE/ACM Int. Conf. Software Engineering (ICSE)*, May 2021, pp. 436–447, DOI 10.1109/ICSE43902.2021.00049 (confirmed via Crossref); arXiv:2102.03300.
- [53] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?," in *Proc. 22nd ACM SIGSOFT Int. Symp. Foundations of Software Engineering (FSE)*, Hong Kong, Nov. 2014, pp. 654–665, DOI 10.1145/2635868.2635929.
- [54] P. Gyimesi, B. Vancsics, A. Stocco, D. Mazinanian, Á. Beszédes, R. Ferenc, and A. Mesbah, "BugsJS: a Benchmark of JavaScript Bugs," in *Proc. 12th IEEE Int. Conf. Software Testing, Verification and Validation (ICST)*,

Apr. 2019, pp. 90–101, DOI 10.1109/ICST.2019.00019 (existence and venue confirmed via Crossref); project site <https://bugsys.github.io/> (dataset details are taken from the site; the paper text was not opened).

[55] S. P. Kumar, S. Bararia, and K. Raj, "Bigger Isn't Always Better: A Comparative Evaluation of LLMs for Automated Code Review," arXiv:2606.15689, 2026 (preprint; **low confidence**: the identifier month (06) conflicts with the stated submission date, 9 April 2026).

[56] J. Isbarov, U. Suleymanov, I. Shumailov, and M. Kantarcioglu, "GitInject: Real-World Prompt Injection Attacks in AI-Powered CI/CD Pipelines," arXiv:2606.09935, 2026 (preprint).

[57] S. Wang, X. Hou, Z. Liu, Y. Zhao, X. Cheng, Q. Zou, X. Zhang, and H. Wang, "Demystifying and Detecting Agentic Workflow Injection Vulnerabilities in GitHub Actions," arXiv:2605.07135, 2026 (preprint).

[58] S. Thornton, "Can Adversarial Code Comments Fool AI Security Reviewers — Large-Scale Empirical Study of Comment-Based Attacks and Defenses Against LLM Code Analysis," arXiv:2602.16741, 2026 (preprint; title of version 1).

[59] G. Benedetti, L. Verderame, and A. Merlo, "Automatic Security Assessment of GitHub Actions Workflows," in *Proc. 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED)*, DOI 10.1145/3560835.3564554; arXiv:2208.03837.

[60] H. Li, H. Zhang, and A. E. Hassan, "The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering," arXiv:2507.15003, 2025 (preprint).

[61] N. Selvanayagam and T. A. Ghaleb, "AI-to-AI Code Reviews of GitHub Pull Requests," arXiv:2608.21311, 2026 (preprint).

Web documentation (not papers)

[W1] GitHub Docs, "Secure use reference" (GitHub Actions), <https://docs.github.com/en/actions/reference/security/secure-use>, accessed 24 September 2026.

Appendix A. Corrections to the earlier edition (summary)

1. **Reference list**: 27 of 30 entries named a first author that does not match arXiv (for example [1] is by H. Guo et al., not "Liu, X."); all corrected above. Titles and identifiers were correct.
2. **"Terragni et al. [21]"** in the body is Duma et al.
3. **[30] versus [10]**: the "dynamic runtime evaluation" future direction is in [10], not [30].
4. **[8]**: $\kappa = 0.75$ validates the LLM-as-judge framework, not human annotation.
5. **[25]**: timing figures corrected (33% review time, 87% waiting time, >60% median resolution time).
6. **[29]**: the agents are Repository Manager, PR Analyzer, Issue Tracker, and Code Reviewer.
7. **[7]**: the pgvector recommendation and the claim that chunk-level embeddings consistently underperform graph-based approaches are not supported by the abstract and are withdrawn as unverified.
8. **[13]**: the statement that SWE-Review requires the reviewer agent to execute tests is not in the abstract and is withdrawn as unverified.
9. **Prompt-injection screening with Semgrep** (earlier §5.2–5.4) was not supported by any source and is withdrawn.
10. **Uniqueness claims** (earlier Table 1 "YES (unique)" and §9.5) are replaced by Section 11.
11. **Fagan's inspection studies** (earlier §1) were cited from general knowledge and are not in the verified corpus; the sentence is removed.